

MICROSOFT AI SUMMER SCHOOL 2026 · FINAL PROJECT

Local RAG Assistant

An Offline Retrieval-Augmented Generation System Running
Entirely on Device

- Foundry Local SDK
- FastAPI
- React
- SQLite
- Qwen LLM



PROBLEM STATEMENT

Why Local AI?

Cloud-based LLMs are powerful — but they come with critical limitations that make them unsuitable for many real-world scenarios.



No Connectivity

Wilderness survival, remote garages, field operations — places where internet access simply doesn't exist.



Data Privacy

Sensitive corporate documents and proprietary manuals cannot be safely uploaded to third-party cloud APIs.



Zero API Cost

No per-token charges. No subscription fees. Once downloaded, the model runs indefinitely on your hardware.

PREVIOUS WORK

Edge AI & On-Device Intelligence

Building on prior experience with NFC-powered, battery-free, AI-enhanced edge devices for electrochemical measurement — demonstrating that powerful AI can run entirely at the edge, without cloud dependency.

1

NFC Potentiostat (BME4496 Graduation Project)

A wireless, battery-free electrochemical measurement platform powered by NFC. Embeds on-device AI for real-time signal processing and analysis — running inference locally without any server or internet.

2

From Hardware Edge AI → Software Edge AI

The NFC Potentiostat proved that AI can operate entirely on constrained edge devices. This RAG project extends that philosophy to software: running LLM inference, embedding generation, and vector search entirely on a local machine using Microsoft Foundry Local.

3

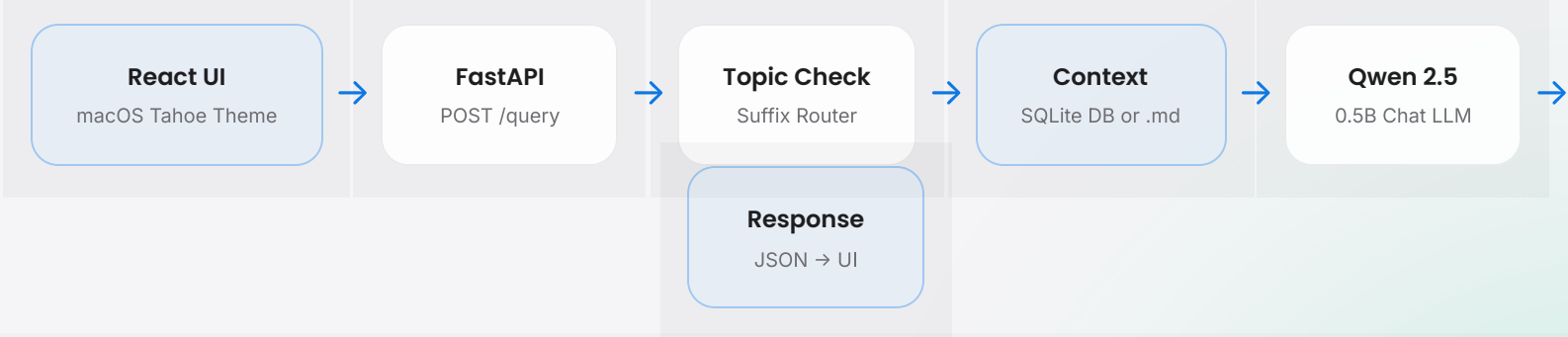
Shared Principles

Both projects share the same core design philosophy: **offline-first, privacy-preserving, zero-latency AI** — whether on an NFC tag or a laptop CPU.

[View NFC Potentiostat Project →](#)

How It Works

A modular pipeline connecting the React UI to local AI models through FastAPI, with dynamic RAM-only context loading.



Dynamic RAM-Only Context

Topic manuals (.md) are loaded into memory per-request and automatically garbage-collected after the response is sent. Zero database pollution.



Cosine Similarity Search

General queries use Qwen Embedding (0.6B) to compute vector similarity against pre-indexed document chunks in SQLite.



Academic Citations

Each topic query automatically appends a formal bibliographic reference (e.g., U.S. Army FM 21-76) so the LLM can ground its answers.

LIVE DEMONSTRATION

DEMO

Three Scenarios

Proving that the system retrieves accurately, cites sources,
and handles uncertainty responsibly.

SCENARIO A — TOPIC RAG

Correct Retrieval + Citation

Toggle "Vehicle Guide" and ask a technical question. The bot answers from the manual and cites the source.

"How do I change a flat tire?"

SCENARIO B — GENERAL RAG

Database Lookup

No toggle active. Ask a general question. The bot retrieves from the SQLite vector database using cosine similarity.

"What is Foundry Local?"

SCENARIO C — FALLBACK

Graceful "I Don't Know"

Ask an out-of-scope question. The bot refuses to hallucinate and states the information is not in the context.

"How do I build a spaceship?"

CLOSING

Lessons Learned & Future Work

Challenges Overcome

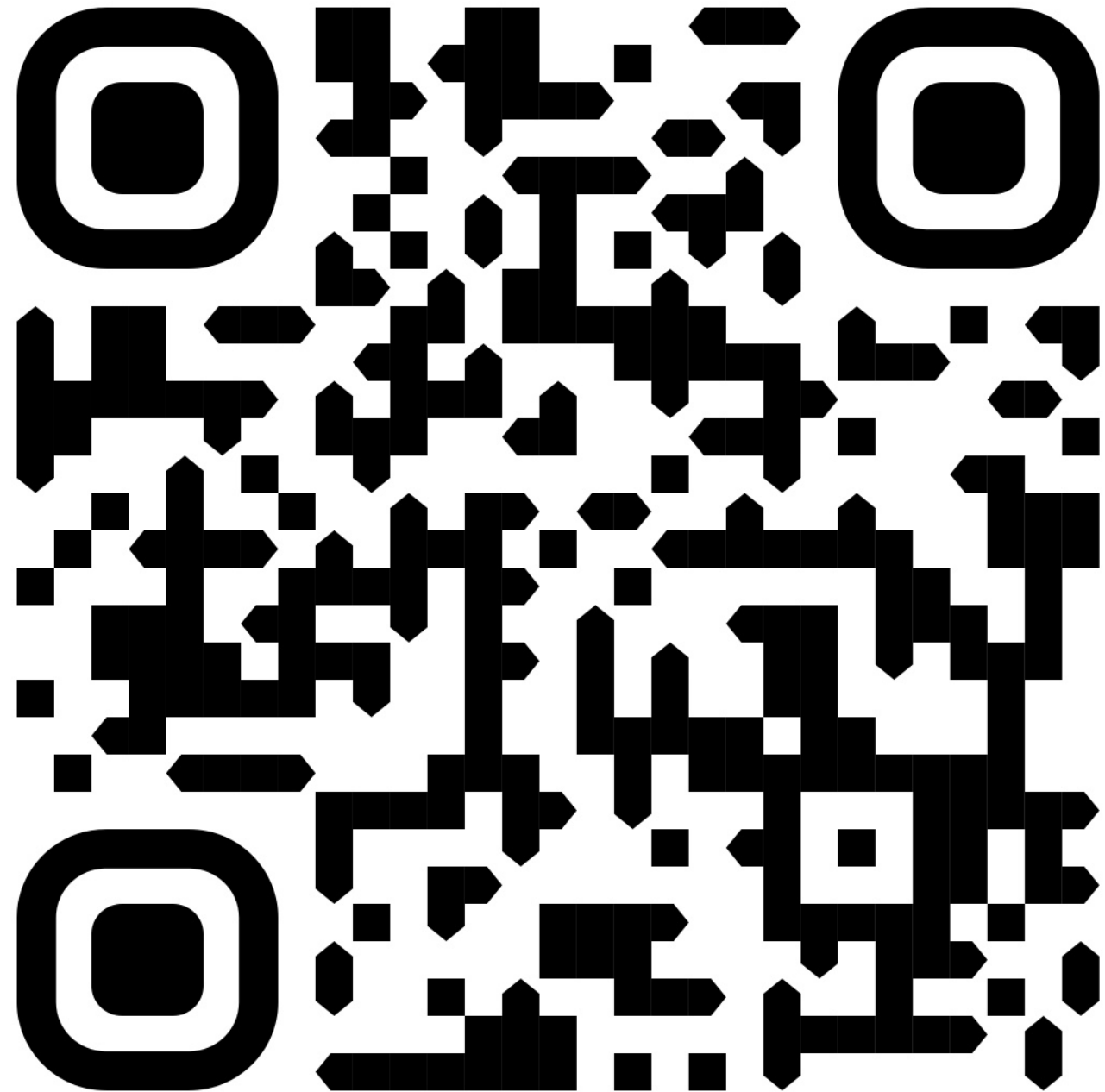
- › CORS configuration between React (port 5173) and FastAPI (port 8000)
- › Python global variable scoping and memory lifecycle management
- › Absolute path resolution for cross-platform file loading
- › SQLite thread-safety with `check_same_thread=False`

Future Improvements

- › Real-time streaming responses (SSE / StreamingResponse)
- › PDF upload endpoint for dynamic document ingestion
- › Larger LLM models for higher accuracy (Qwen 7B, Phi-3)
- › Persistent chat history across browser sessions

Thank You

Questions? → efeates3300@gmail.com



LINK TO MY PORTFOLIO

<https://efeatesh.github.io/Portfolio/>